OOP

Build a Class With a Partner

Module 1: Python Foundations

```python
class BoardGame:
    def __init__(self, n):
        self.players = n
uno = BoardGame(8)
print(uno.players)  # 8
```

By the End of Today, You'll Be Able To…

1

Define a class with a method, from a template

2

Confirm each object stores its own data

Partner Up for Today

Today’s exercise is explicitly collaborative — find your partner before the code-along starts.

Reading Recap: Meet the Objects

Today's Reading: Defining a Class From Scratch

A budgeting app saves for several goals at once, like a Bike Fund and a Trip Fund. One goal's numbers must never mix with another goal's.
Reopen the reading:
Meet the Objects: Defining a Class From Scratch

The Scenario: A Budgeting App's Savings Goals

One goal

Bike Fund has a 500 target and 420 saved. Three variables hold it: `name`, `target_amount`, `saved_amount`.

Ten goals

`name_2`, `target_2`, `saved_2`, and so on, and hope you never mix them up. The plan falls apart.

One class

`SavingsGoal` is written once. Bike Fund (420 of 500) and Trip Fund (1,200 of 1,200) each keep their own values.

READING RECAP

Key Terms and Code From the Reading

Class

`SavingsGoal` describes every goal, written once.

Object

`goal_a` is one goal built from the class: an instance.

`__init__`

Runs once per new object and stores its starting values.

Attribute

`goal_a.name` is a value stored on one object.

Method and `self`

`goal_a.is_funded()` runs with `self` set to `goal_a`.

READING CHECK

`goal_b` holds 1200. After `goal_a.saved_amount = 480`, what is `goal_b.saved_amount`?

A `480`

B `1200`

C `1680`

ANSWER KEY

Answer Key: Each Object Keeps Its Own Values

Correct answer: B. `1200`

`goal_a.saved_amount = 480` changes only `goal_a`. `goal_b` comes from the same class, but it stores its own attributes, so it still holds 1200.

READING CHECK

`is_funded()` checks saved >= target. `goal_a` has 420 of 500 saved. What does `goal_a.is_funded()` return?

A `False`

B `True`

C `420`

ANSWER KEY

Answer Key: `self` Is the Object Before the Dot

Correct answer: A. `False`

In `goal_a.is_funded()`, `self` is `goal_a`, so it checks 420 >= 500, which is False. The same call on `goal_b` checks 1200 >= 1200: True.

Your Questions From the Reading

Pair up

1 min: tell your partner one question you still have.

Ask

We take 2 or 3 questions now.

Park it

Bigger ones wait for today's matching slide.

A Class Bundles Data and Behavior Together

DATA

title
count
category

A Class Bundles Data and Behavior Together

DATA

title
count
category

BEHAVIOR

A way for the game to self-check.

One Blueprint Can Build Many Independent Objects

CLASS

The blueprint: defines what every house has.

One Blueprint Can Build Many Independent Objects

CLASS

The blueprint:
defines what every house has.

OBJECT

One real house:
its own address and furniture.

One Blueprint Can Build Many Independent Objects

CLASS

The blueprint: defines what every house has.

OBJECT

One real house: its own address and furniture.

INDEPENDENT

Another house: same blueprint, different stuff.

CHECK FOR UNDERSTANDING

In the blueprint analogy, what is an object?

A The blueprint itself

B One actual house built from it

C The architect

ANSWER KEY

Blueprint Analogy Recall

Correct: One actual house built from it

The blueprint is the class — the pattern. Each real house built from it, with its own address and contents, is an object.

Five New Terms, One Object Model

Class

blueprint

Object

instance

`__init__`

setup at creation

Attribute

stored data

`self`

this object

CHECK FOR UNDERSTANDING

Inside a method, what does `self` refer to?

A The class itself

B A random object

C The specific object the method was called on

ANSWER KEY

`self` Recall

Correct: The specific object the method was called on

`self` tells a method which object's own data to use. Call the method on another object, and `self` points there instead.

IN THE NEWS

Gene Names Slid **One Row** Off Their Data

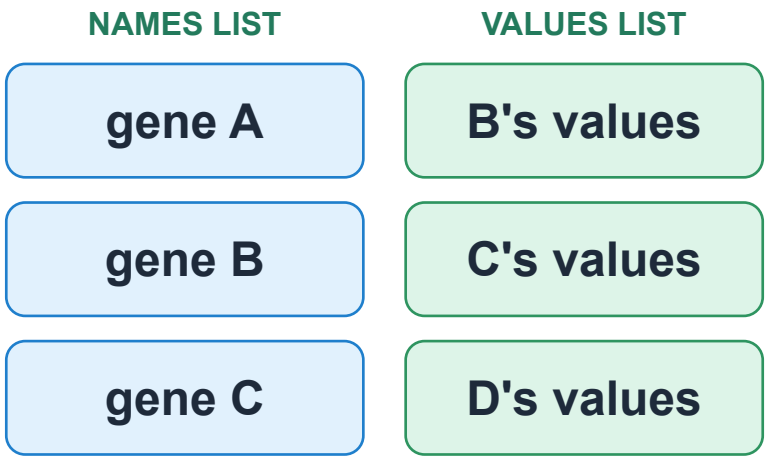


Illustration: names one row off their values

Statisticians found gene names one row off
They also found drug labels flipped
Duke ended trials that enrolled 110 patients

Names in one list, numbers in another. What keeps each pair lined up?

Source: Baggerly & Coombes, Annals of Applied Statistics, 2009

CODE ALONG

Open the Code-Along

Open this file:
`Module1_Lesson13_oop-building-a-class-with-a-partner_STUDENT.py`
Run it after every section: **`python3`** + the file name

Sections, in order:
1. Step 1: **`__init__`** and Attributes · one partner types, one reads each line
2. Step 2: Add a Method · give the object something to do with its data
3. Step 3: Prove Independent Storage · does a second object keep its own data?

Step 1: `__init__` and Attributes

```python
class BoardGame:
    def __init__(self, title, players_available,
                 category):
        self.title = title
        self.players_available = players_available
        self.category = category

connect4 = BoardGame("Connect Four", 4, "Strategy")
print(connect4.title)
print(connect4.players_available)
```

Step 2: Add a Method

```python
class BoardGame:
    def __init__(self, title, players_available,
                 category):
        self.title = title
        self.players_available = players_available
        self.category = category

    def is_low_stock(self):
        return self.players_available < 3
connect4 = BoardGame("Connect Four", 4, "Strategy")
print(connect4.is_low_stock())
```

Step 3: Prove Independent Storage

```python
connect4 = BoardGame("Connect Four", 4, "Strategy")
uno = BoardGame("Uno", 8, "Party")

connect4.players_available = 1
print(connect4.is_low_stock())
print(uno.is_low_stock())
```

CODE-ALONG DONE

Back to the Slides: Review

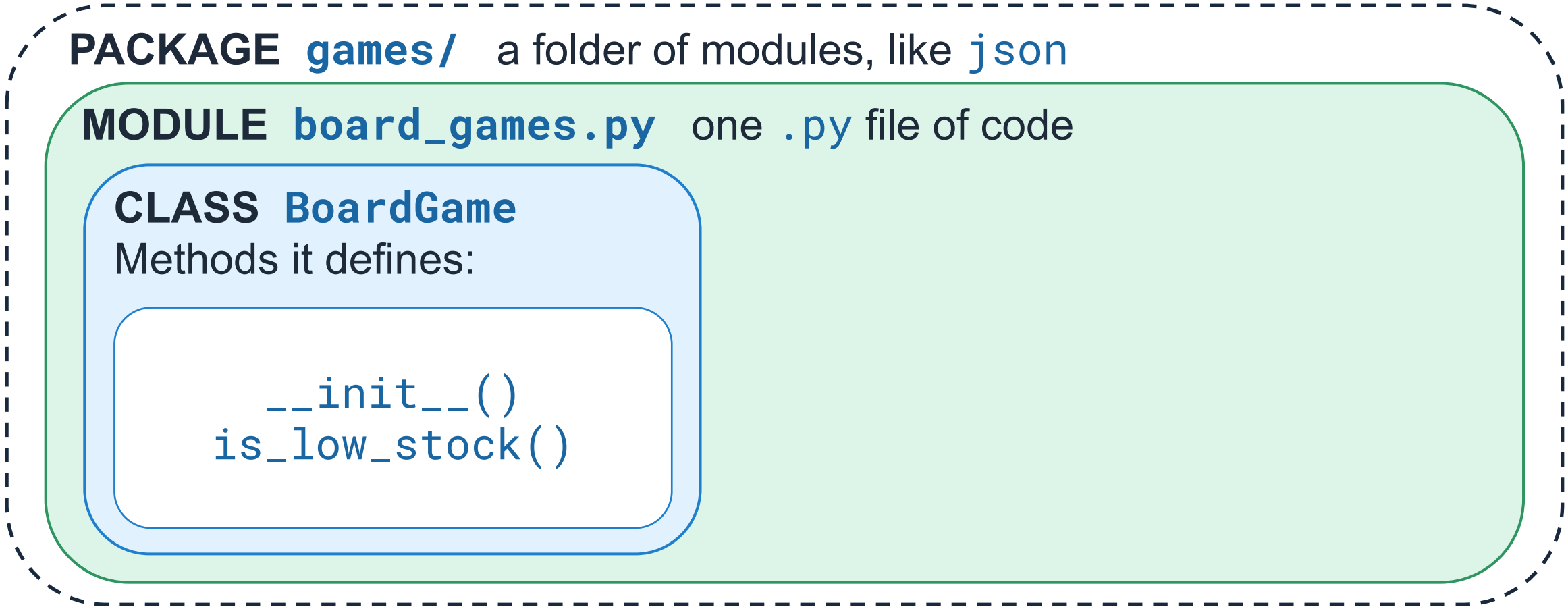
You just built a class. First see where it lives, then check that it stuck.

Where Today's Class Actually Lives

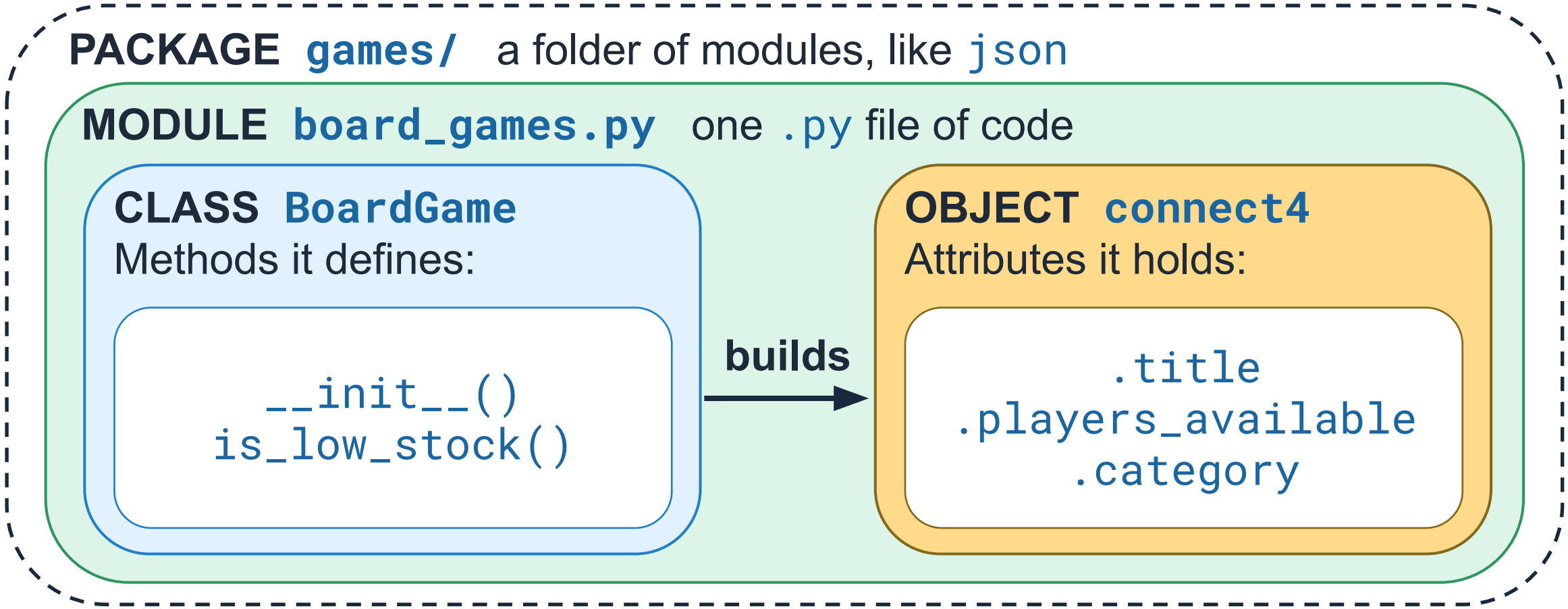
PACKAGE `games/` a folder of modules, like `json`

MODULE `board_games.py` one `.py` file of code

Where Today's Class Actually Lives



Where Today's Class Actually Lives



CHECK FOR UNDERSTANDING

Which is which? Label `json`, `BoardGame`, and `.title`, in that order.

A

class
object
method

B

object
module
attribute

C

package
class
attribute

ANSWER KEY

Which Is Which, Confirmed

Correct: C. package, class, attribute

`json` is a folder of modules. `BoardGame` is the class, the blueprint.
`.title` is data stored on one object, so it is an attribute.
`is_low_stock()` would be the method.

QUICK CHALLENGE

In the Step 1 code, why does `connect4.players_available` hold 4?

A Arguments fill parameters in order, after `self`

B Python matches values to names by type

C 4 is a built-in default value

ANSWER KEY

Step 1 Trace

Correct: Arguments fill parameters in order, after `self`

`self` fills in automatically, so `"Connect Four"` lands in `title`, 4 in `players_available`, `"Strategy"` in `category`.

QUICK CHALLENGE

In the Step 2 code, which change makes `connect4.is_low_stock()` return `True`?

A Set the player count to 2

B Set the category to “Party”

C Set the title to “Uno”

ANSWER KEY

Step 2 Trace

Correct: Set the player count to 2

`is_low_stock()` only reads `players_available`. `2 < 3` is `True`.
The title and category never enter the check.

QUICK CHALLENGE

In the Step 3 code, why does `uno.is_low_stock()` still return `False`?

A The `uno` check never ran

B Python undid the change to `connect4`

C Each object keeps its own attributes

ANSWER KEY

Independent Storage, Confirmed

Correct: Each object keeps its own attributes

Changing `connect4.players_available` touches only `connect4`. `uno` still holds its own 8, and `8 < 3` is `False`.

Watch It Run: Two Independent Objects

Step 1 of 5

connect4 is built. connect4.players_available is 4

```
connect4 = BoardGame("Connect Four", 4, "Strategy")
uno = BoardGame("Uno", 8, "Party")

connect4.players_available = 1
print(connect4.is_low_stock())
print(uno.is_low_stock())
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

`__init__` Runs Once — at Object Creation

CREATE

`BoardGame(...)`
calls **`__init__`**.

`__init__` Runs Once — at Object Creation

CREATE

`BoardGame(...)` calls `__init__`.

STORE

Attributes are saved on that object.

`__init__` Runs Once — at Object Creation

CREATE

`BoardGame(...)` calls `__init__`.

STORE

Attributes are saved on that object.

READ

`connect4.title` later just reads the stored value.

QUICK CHALLENGE

Does `__init__` run again every time `connect4.title` is read?

A Yes — every time

B No — the stored value is just read

C It raises an error

ANSWER KEY

`__init__` Misconception Check

Correct: Nothing re-runs — the stored value is just read

`__init__` runs once when `BoardGame(...)` creates the object. Later attribute access reads what was stored then.

This Is the Same Shape Your Project Uses

TODAY

`BoardGame` class:
real attributes + methods

PROJECT

`Item` class:
the same object-oriented shape

TIP

“One Object per Game, or Three Parallel Lists?”

OBJECTS

5 games as 5 independent objects.

PARALLEL LISTS

```
titles = []
counts = []
categories = []
```

Answer like an interview, with the Tradeoff Lens: option, benefit, cost, when you'd choose it.

MODEL ANSWER

Model Answer: Tradeoff Lens

OPTION

One `BoardGame` object per game, data and methods together.

BENEFIT

A game's title, count, and category can't drift apart.

COST

A little more code up front than three lists.

WHEN I'D CHOOSE IT

Records with several fields, since parallel lists fall out of sync.

Key Terms — Recap

Class

blueprint

Object

instance

`__init__`

setup at creation

Attribute

stored data

`self`

this object

So Far, We've Covered…

TODAY

A real class, built with a partner.

PROJECT

The project's `Item`-equivalent class.

NEXT

What `__str__` and `__repr__` actually do.

Watch: OOP in 7 minutes